BANGLADESH UNIVERSITY OF PROFESSIONALS
Military Institute of Science and Technology
B.Sc. in Computer Science and Engineering
Level-3, Term-I Final Examination : June 2016

Subject: CSE – 303 (Compiler)

Time: 3.00 hours **Full Marks: 210**

INSTRUCTIONS:

a. Use **SEPARATE** answer scripts for each section.
b. Answer any **THREE** questions from each section.
c. Figures in the margin indicate full **marks**.
d. Assume reasonable data if necessary.
e. **Symbols** used have their usual meanings.

SECTION-A

Question – 1

a. Which stage in a compiler is most likely to save line numbers of identifiers in the symbol table? Is any other stage suitable for this purpose? Explain clearly. **5+6=11**

b. What is **"panic mode"** recovery in compiler? **07**

c. Formulate regular definition for **"number"** tokens in a certain programming language. The numbers in this language consist of three parts in the same sequence as detailed below: **17**

Integer part: Consists of any non-zero number of digits from 0 to 9. Cannot start with a 0, unless this 0 is the only digit in this part.

Decimal part: This part is optional. Consists of decimal point followed by any non-zero number of digits from 0 to 9. Cannot end with a 0. Thus, only a single zero in this part is not permitted.

Exponent part: This part is optional. Consists of character 'E' followed by an optional minus (-) sign, which is followed by any non-zero number of digits from 0 to 9.

A few valid numbers are:
123, 123.456, 0.456, 123.456E789, 123.456E-789, 123E789, 123E000.

A few invalid numbers are:
0123, 123.0, 123.456E+789

Question – 2

a. What are **sentinels**? Describe where sentinels are used in buffer pairs in lexical analysis. How does the use of sentinels speed up the input process? **3+3+7=13**

b. **"The token name influences parsing decisions, while the attribute value influences translation of token after the parse"** – explain clearly with necessary examples. **07**

c. The following table shows a subset of the unary and binary operators used by the C++ language. Corresponding token names are also shown in the table. The attribute value column has been intentionally left blank. Fill up the attribute value column where necessary. Draw a transition diagram which recognizes the tokens. **15**

Lexeme	Token Name	Attribute Value
>	relop	
<	relop	
>=	relop	
<=	relop	
&&	logop	
~	bitop	
&	bitop	
^	bitop	
<<	bitop	
>>	bitop	

Question – 3

a. Eliminate left recursion from the following grammar: **20**

$$S \rightarrow Sc \mid Ta \mid b$$
$$T \rightarrow Tc \mid Sd \mid d$$
$$U \rightarrow Ua \mid Td \mid Sb \mid b$$

b. Comment with reasoning on whether the following grammars are ***LL(1)*** or not. **15**

i) $S \rightarrow ABc, \quad A \rightarrow a \mid \epsilon, \quad B \rightarrow b \mid \epsilon$
ii) $S \rightarrow ACB, \quad A \rightarrow a \mid B \mid \epsilon, \quad B \rightarrow b \mid \epsilon, \quad C \rightarrow c \mid \epsilon$
iii) $S \rightarrow X, \quad X \rightarrow aXBA \mid bAXB \mid c, \quad A \rightarrow a, \quad B \rightarrow a \mid \epsilon$
iv) $A \rightarrow C \mid \epsilon, \quad C \rightarrow CE, \quad C \rightarrow \epsilon, \quad E \rightarrow \epsilon$
v) $S \rightarrow S(S) \mid \epsilon$

Question – 4

a. Answer the following questions for predictive parsing. **10+5+5=20**

i) During construction of a parsing table ***M***, for each production rule $A \rightarrow \alpha$ of the grammar, if ϵ is in ***FIRST (α)***, we add $A \rightarrow \alpha$ to ***M[A, b]***, for each terminal ***b*** in ***FOLLOW(A).*** Why? Explain clearly in light of the parsing mechanism.

ii) During a parsing, if w is the input that has been matched so far, and ***S*** is the start symbol for the corresponding grammar, with $S \Rightarrow^* w\alpha\beta\gamma$, what will be the stack contents? Also comment on the nature of symbols in **w** and stack contents regarding being terminals/non-terminals.

iii) During parsing, if the input pointer is pointing to the symbol ***a***, and the top stack symbol is ***X***, with the entry ***M[X,a]*** being $X \rightarrow UVW$, explain how the stack contents will be altered. Why?

b. Left factor this grammar: **15**

$$S \rightarrow T : S \mid \epsilon$$
$$T \rightarrow UR$$
$$R \rightarrow * T \mid \epsilon$$
$$U \rightarrow x \mid y \mid [S]$$

Hence, find the **FIRST** and **FOLLOW** sets for each non-terminal in the grammar obtained.

SECTION-B

Question 5

a. Define **S-attributed** and **L-attributed** syntax directed definitions (SDD). Elaborate on each with necessary examples and counter-examples. How are attributes evaluated in each case? **20**

b. Present the **S-attributed** syntax directed definition which constructs syntax trees for a simple expression grammar involving only the binary operators + and -. Then show the annotated parse tree and syntax tree for **α – 4 + C**. **15**

Question 6

a. Present the SDD that takes a simple declaration consisting of a basic type followed by a list of identifiers and stores the identifier types in a symbol table. Why is it imperative that inherited attributes are needed for this SDD? What is the deficiency (if any) of the SDD you have presented? **20**

b. Show how the SDD in **Question 6(a)** will work on the declaration, **15**

float id1, id2, id3.

Question 7

a. Draw syntax tree for the expression: **15**

a + (a * a – a) * (a * a – a) * (a * a + a) – a.

And hence, convert the syntax tree into a DAG.
Consider usual precedence and associativity of operators.

b. Use value-number method to store the DAG in **Question 7(a)**, in an array of records. Then, represent the same DAG using three address code. How can we speed up this value-number method construction? Explain. **5+5+10=20**

Question 8

a. Present the syntax directed translation (SDT) scheme that computes types and their widths for basic and array types. Hence, show the parse tree for the type **int [2][3].** **20**

b. Find out the three address code for the statement, **15**

n = g (f(a), a, b[i], a*2).

Assume any missing information within reasonable limits. Do not use any unnecessary instruction.

BANGLADESH UNIVERSITY OF PROFESSIONALS
Military Institute of Science and Technology
B.Sc. in Computer Science and Engineering
Level-3, Term-I Final Examination: May-June 2017

Subject: CSE-303, Compiler

Time: 3.00 hours **Full Marks: 210**

INSTRUCTIONS:

a. Use **SEPARATE** answer scripts for each section.
b. Answer any **THREE** questions from each section.
c. Figures in the margin indicate full **marks**.
d. Assume reasonable data if necessary.
e. **Symbols** used have their usual meanings.

SECTION-A

Question – 1

a. Explain in detail the following statements. **5+10=15**

i) "Java language processors combine compilation and interpretation".

ii) "The front end of a compiler is concerned about the source language, while back end of compiler is concerned about the target machine. The intermediate representation, between the two, is concerned neither about the source language nor about the target language".

b. Explain clearly, with necessary justifications, the lexical errors (if any) that will occur for each of the following C/C++ statements. **20**

You need to consider both the cases where the lexical analyzer uses and does not use panic mode recovery strategy. Then provide the list of tokens that may be generated in both the cases. You may use your own token names.

i) int float double int float float;
ii) ;) (} {))))) cse dhaka mist
iii) int 12345 cse, dhaka;

Question – 2

a. After you learned defining tokens using regular definitions, some of your friends in haste, come up with the following definitions for identifiers and numbers: **20**

i) id → letter (letter*|digit*)
ii) id → letter (letter digit)*
iii) id → letter * (letter |digit)*
iv) number → digit + (. digit +) (E[+-]? digit+)?
v) number → digit + (. digit)? (E+-? digit+)?

All of the above definitions are incorrect. Explain in each ease what is wrong with the definition.
Also for each case, show,

i) One example string which matches with both the correct and incorrect definition,
ii) One example string which is rejected by the correct definition but matches the incorrect definition.

If in some case any of the above mentioned examples is not possible you need to point out that as well.
You may assume that letter and digit have been correctly defined.

b. Write a Yacc plus Lex program which calculates the value of an arithmetic expression which involves single line inputs only. The arithmetic expression comprises of numbers with decimal points, addition, subtraction, multiplication and division operators. Parentheses can be used in the expression. The expressions may also contain the sin, cos and tan trigonometric functions. **15**

Question – 3

a. Explain in detail the relative advantages and disadvantages of having individual and class tokens for the operators. **10**

b. Comment with reasoning on whether the following grammar is LL (1) or not. **5**

S→ ABc, A→ a | €, B→ b|€

c. Eliminate left-recursion from the following grammar: **20**

S→ Sc | Ta | Ub |b
T→ Tc | Sd | d
U→ Ua | Td | Sb | b

Question – 4

a. Answer the following questions for predictive parsing. **7+6+7=20**

i) During construction of a parsing table M, for each production table A→ α of the grammar, if € is in FIRST (α), we add A→ α to M [A, b], for each terminal b is FOLLOW (A). Why?
Explain clearly in light of the parsing mechanism.

ii) What does multiple entries in a parsing table mean? Is the corresponding grammar LL (1)?

iii) During parsing, of the input pointer is pointing to the symbol a, and the top stack symbol is X, with the entry M [X, a] being X→ UVW, explain how the stack contents will be altered. Why?

b. Construct procedure based recursive – descent parsers staring with each of the following grammars: **15**

i) S→ S (S) S | €
ii) S→ 0S1 | 01

SECTION-B

Question 5

a. Present the S-attributed syntax directed definition (SDD) which constructs syntax trees for a simple expression grammar involving only the binary operators + and - . Then show the annotated parse tree and syntax tree for **a–4+c.** **15**

b. Show how the SDD in Question 5 (a) changes when left recursion is eliminated from the corresponding grammar. Why is it imperative that we use inherited attributes here?
Show the dependency graph for **a-4+c**. **20**

Question 6

a. Construct syntax tree and hence DAG for the expression, **15**

((x + y) – ((x + y) * (x – y))) + ((x + y) * (x – y).

Then store the DAG using value number method. Assume usual precedence and associativity of operations.

b. Find out the three address code for the statement, **10**

n = g (f (a), a, a * 2).

Assume any missing information within reasonable limits. Do not use any unnecessary instruction.

c. Explain how control flow statements are handled in SSA. **10.**

Question 7

a. Describe the contents of an activation record with necessary figures. **15**

b. We have the given C program and accompanying functions as shown in code1 and code2. **20**
For a certain situation, the sequence of function calls and returns are as given.
Now for the scenario when the call is in line 6,

i) Draw the activation tree.
ii) Sketch the activation records.

Question 8

a. For the given code as shown in code 3, generate machine code for stack allocation sizes of activation records for procedures p, q, and r are 20, 80 and 40 bytes respectively. Code for these procedures start at addresses 1000, 2000, and 4000 respectively. Each action requires 20 bytes. Each word in 4 bytes. Stack starts at address 8000. **20**

b. For the given code as shown in code 4 , identify the basic blocks. And hence draw the flow graph. **15**

```c
/* main.c */
#include <stdio.h>

void main() {
  int i, j, k;
  float a[20];
  /* .......... */
  i = fone(a[5], a[15]);
  /* .......... */
  i = fone(a[5], a[15]);
  j = ftwo(i, k);
  printf("The results, i = %d, j = %d", i, j);
}
```

```c
/* fone.c */
int fone(m1, m2) {
  int f1, ii, jj = 0;
  for (ii = 1; ii <= m2; ii++)
    {
      jj = ftwo(m1);
    }
  f1 = jj;
  return f1;
}
```

```c
/* ftwo.c */
float ftwo(n) {
  float n1, n2;
  n2 = n / 2;
  if (n2 > 10)
    n1 = ftwo(n2);
  return n1;
}
```

Code1: C program for Question 7 (b).

```
1                    enter main()
2                    enter fone(10,-5)
3                    leave fone(10,-5)
4                     enter fone(30,1)
5                        enter ftwo(30)
6                           enter ftwo(15)
7                           leave ftwo(15)
8                        leave ftwo(30)
9                     leave fone(30,1)
10                   leave main()
```

Code 2: Sequence of function calls and returns for Question 7 (b).

```
// Code for p
action1
call q
action2
call q
halt
// Code for r
call r
action7
action8
call r
return
```

```
// Code for q
action3
call q
call r
action4
call r
action5
call r
action6
return
```

Code 3: Code for Question 8 (a).

```
1)    i = 1
2)    j = 1
3)    t1 = 10 * i
4)    t2 = t1 + j
5)    t3 = 8 * t2
6)    t4 = t3 - 88
7)    a[t4] = 0.0
8)    j = j + 1
9)    if j <= 10 goto (3)
10)   i = i + 1
11)   if i <= 10 goto (2)
12)   i = 1
13)   t5 = i - 1
14)   t6 = 88 * t5
15)   a[t6] = 1.0
16)   i = i + 1
17)   if i <= 10 goto (13)
```

Code 4: Code for Question 8 (b).

9

BANGLADESH UNIVERSITY OF PROFESSIONALS
Military Institute of Science and Technology
B.Sc. in Computer Science & Engineering
Level-3,Term-I Final Examination : June- July 2018

Subject: CSE-303, Compiler Design

Time: 3.00 hours **Full Marks: 210**

INSTRUCTIONS:

a. Use **SEPARATE** answer scripts for each section.
b. Answer any **THREE** questions from each section out of **FOUR.**
c. Figures in the margin indicate full **marks**.
d. Assume reasonable data if necessary.
e. **Symbols** used have their usual meanings.

SECTION-A

Question – 1

a. What is top-down parsing? Write recursive procedures for top-down parsing for the following grammar: **12**

$$S \rightarrow cAd$$
$$A \rightarrow ab|a$$

b. Briefly explain how the order in which alternates are tried can affect the language accepted for top-down backtracking parsers with an illustrative example. **8**

c. What is indirect left-recursion? Show that the following grammar is indirectly left recursive. Also, eliminate left-recursion entirely from this grammar. **15**

$$A \rightarrow Bxy|x$$
$$B \rightarrow CD$$
$$C \rightarrow A|a$$
$$D \rightarrow b$$

Question – 2

a. What is left-factoring? Left-factor the following grammar: **8**

$$S \rightarrow iCtS|iCtSeS|a$$
$$C \rightarrow b$$

b. Briefly describe the model of a non-recursive predictive parser and the predictive parsing program. **12**

c. Consider the following grammar and the corresponding parsing table: **15**

$$E \rightarrow TE'$$
$$E' \rightarrow +TE'|\epsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow * FT'|\epsilon$$
$$F \rightarrow (E)|id$$

	id	$+$	$*$	$($	$)$	$\$$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow * FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

With input $id + id * id$, show the sequence of moves by the predictive parser.

Question – 3

a. Briefly describe different parsers of a compiler with appropriate examples for each phase. **16**

b. What is the difference between an interpreter and a compiler? What is a hybrid compiler? **7**

c. Write the rules for constructing regular expressions over an alphabet. **8**

d. What is the regular expression for denoting all strings whose length is not two? Assume a binary alphabet. **4**

Question – 4

a. Formulate regular definition for "number" tokens in a certain programming language. The numbers in the language consists of three parts in the same sequence as detailed below: **17**

Integer part: Consists of any non-zero number of digits from 0 to 9. Cannot start with a zero, unless this zero is the only digit in this part.

Decimal part: This part is optional. Consists of decimal points followed by any non-zero number of digits from 0 to 9. Cannot end with a 0. Thus only a single zero in this part is not permitted.

Exponent part: This part is optional. Consists of character 'E' followed by an optional minus (-) sign, which is followed by any non-zero number of digits from 0 to 9.

A few valid numbers are:
123, 123.456, 0.456, 123.456E789, 123.456E-789, 123E789, 123E000.

A few invalid numbers are:
0123, 123.0, 123.456E+789

b. What is the role of a transition diagram in the construction of a lexical analyzer? Show the transition diagram for the relational operators: <, <=, =, <>, >, >=. **10**

c. What are sentinels? Describe where sentinels are used in buffer pairs in lexical analysis. **8**

SECTION-B

Question – 5

a. Define SDD. Define S-attributed and L-attributed syntax directed definitions (SDD) with necessary examples. Determine whether following SDD is S-attributed or L-attributed or not. Justify your answer. **3+6+6 +5=20**

PRODUCTION	*SEMANTIC RULE*
$X \rightarrow ABCDE$	$C.c = func(X.x, A.a, E.e)$

b. For the following grammar find out the parse tree for an input sentence "132". Then from the given SDD, annotate the parse tree by calculating the values of the attributes at the various nodes. Then construct the dependency graph for the same input sentence "132". Carry out topological sorting of this graph. **5+5+ 5=15**

PRODUCTION	SEMANTIC RULE
$S \rightarrow AB$	$S.x = A.x + B.z$
$A \rightarrow A_1C$	$A.x = A.z + C.z$ $A.z = A_1.x + 10$ $C.z = A.z + A_1.x + C.x$
$A \rightarrow F$	$A.x = 1 + F.g$
$B \rightarrow F$	$B.x = 2 + F.g$ $B.z = B.x + 1$
$C \rightarrow F$	$C.x = 3 \times F.g$
$F \rightarrow 1$	$F.g = 1$
$F \rightarrow 2$	$F.g = 2$
$F \rightarrow 3$	$F.g = 3$

Question – 6

a. Find out the three address code, quadruples and triples representation for the following expression: **15**

$$x = y * z * \frac{p}{b} + a + d - q$$

b. Construct the syntax tree and DAG for the following expression: **5+5=10**

$$b = g * h - \frac{g}{h} - c * e \; - c * e$$

c. Define the value number method for constructing DAG's with necessary examples. **10**

Question – 7

a. Define activation record. Why activation record is used? Describe the contents of an activation record with necessary figures and examples. **2+3+10=15**

b. Explain how control flow statements are handled in SSA. **10**

c. Describe any two characteristics of peephole optimization with necessary examples. **5+5=10**

Question – 8

a. For the following code, generate machine code using static allocation for procedures a, b and c. Assume that code for procedures a, b and c start at the addresses 100, 300 and 500 respectively. Also assume that activation records for procedures a, b and c start at the addresses 700, 900 and 1100 respectively. Further assume that each ACTION instruction takes 32 bytes. **15**

```
// code for a
action1
action2
call b
action3
halt

// code for b
action4
call c
return

//code for c
action5
action6
action7
return
```

b. Partition the following three address instructions into basic blocks: **8**

1) $i = 1$
2) $j = 1$
3) $t1 = 10 * i$
4) $t2 = t1 + j$
5) $t3 = 8 * t2$
6) $t4 = t3 - 88$
7) $a[t4] = 0.0$
8) $j = j + 1$
9) $if\ j \leq 10\ goto\ (3)$
10) $i = i + 1$
11) $if\ i \leq 10\ goto\ (2)$
12) $i = 1$
13) $t5 = i - 1$
14) $t6 = 88 * t5$
15) $a[t6] = 1.0$
16) $i = i + 1$
17) $if\ i \leq 10\ goto\ (13)$

c. Find out the machine instructions for the following three-address statements: **4× 3=12**

i. $x = y - z$
ii. $a[j] = c$
iii. $x = * p$

BANGLADESH UNIVERSITY OF PROFESSIONALS
Military Institute of Science and Technology
B.Sc. in Computer Science and Engineering
Student Group: 72 < Earned Credit Hour <= 108, Spring Final Examination: June 2019

Subject: CSE-303, Compiler

Time: 3.00 hours **Full Marks: 180**

INSTRUCTIONS:

a. Use **SEPARATE** answer scripts for each section.
b. **Question-1 in Section A** and **Question-8 in Section B** are compulsory.
c. Answer **any OTHER TWO** questions from **EACH SECTION**.
d. Figures in the margin indicate full **marks**.
e. Assume reasonable data if necessary.
f. **Symbols** used have their usual meanings

SECTION-A

Question – 1

a. Differentiate between compiler, assembler and interpreter. Define "Hybrid Compiler" and justify the reason behind using a **just-in-time** compiler with an example. **04+06 =10**

b. Identify the significance of a symbol table in a compiler. Demonstrate how source code is translated into a target code in a compiler using the **input statement** given below. **02+10 =12**

Input statement:

Result = score1 * 100 + score2 * 100

[You must show all the phases of a compiler sequentially and the possible input and output of each phase for the given input statement in a diagram]

c. State the possible error-recovery actions in Lexical Analysis. Explain "Panic mode recovery" with an example. **08**

Question – 2

a. What is "Sentinel"? Define where sentinels are used and how does the use of sentinel speed up the scanning process? **02+04 =06**

b. Tokenize the given source program using the token patterns given in Table 2(b). Also draw the symbol table along with the attributes stored in it. Handle "Lexical Errors" accordingly. **14**

Token	Attributes	Pattern	Sample Lexemes
Keyword	Lexeme itself	Characters **i, f**	if
		Characters **e, l, s, e**	else
		Characters **m, a, i, n**	main
		Characters **p, r, i, n, t, f**	printf
NumType	%d	Characters **i, n, t**	int
	%f	Characters **f, l, o, a, t**	float
	%lf	Characters **d, o, u, b, l, e**	double
Literal	Pointer to the symbol table	Characters surrounded by " and "	"Hello world"
Operator	Lexeme itself	== or < or > or <= or >= or !=	==, !=
Bracket	Lexeme itself	(or) or { or }	(,) , { , }
Id	Pointer to the symbol table	Letters followed by letters or digits or underscores (_)	Name, var11, new_var

Number	Lexeme itself	Any numeric constants; allows decimal and exponential. Doesn't start with 0 unless the number itself is 0 or there is a decimal point after that 0	12, 123.5, 12E5, 12E-5, 123.3+E4, 0, 0.556

Table 2(b)

N.B.

1. Example of 'Lexeme itself ' attribute:
   < Bracket, (> , < Keyword, else >
2. Example of 'Pointer to symbol table' attribute:
   < Id , 3556 >

[Here, **3556** represents the memory address of the symbol table where the attributes of the particular token are sorted]

Source Code to "Find the Largest Number among three numbers" is given below:

```
#include <stdio.h>
int main( )
{
   double n1, n_2, n3;
   int 1st_var;

   printf("Enter three different numbers: ");
   scanf("%lf %lf %lf", &n1, &n_2, &n3);

   if( n1>=n_2 && n1>=n3 )
      printf("%f is the largest number", n1);

   if( n_2>=n1 && n_2>=n3 )
      printf("%f is the largest number", n_2);

   if( n3>=n1 && n3>=n_2 )
      printf("%f is the largest number", n3);

   return 0;
}
```

c. Formulate regular definition and show the transition diagram for the following tokens given below [No explanation required]. **05=05 =10**

(i) Token **Number** :

Pattern: Consists of any non-zero number of digits (0-9). Doesn't start with a zero unless the number itself is 0. Allows decimal points followed by any non-zero number of digits. There must be at least one digit before the decimal number. Also, the decimal part cannot end with a zero. Exponential not allowed.

Example:

Valid: 123, 123.45, 0.78, 0, 15.600007
Invalid: 123E-5, 34E65, 0.0, 67.890, .325

(ii) Token **RelationalOperator**:

Pattern: = , < , > , < = , > = , != , < >

Example:

Valid: < > , != , =
Invalid: << , = = , = <

Question – 3

a. What is top-down parsing? Left factor the following grammer. **02+03 =05**

$S \rightarrow \mathbf{ab}B \mid \mathbf{abc}C \mid \mathbf{abcd}D$
$B \rightarrow \mathbf{b} \mid \in$
$C \rightarrow \mathbf{c} \mid \in$
$D \rightarrow \mathbf{d}$

b. Grammer: **10+05 =15**

```
stmt      : expr
          | for ( opt_expr; opt_expr; opt_expr ) stmt
          | if ( expr ) stmt
          | while ( expr ) stmt
          | return expr ;
Opt_expr : expr | ∈
expr      : id = expr | expr - term | expr + term
          | expr relop expr | term
term      : term * factor | term / factor | factor
relop     : = = | < = | < | > = | > | !=
factor    : ( expr ) | id | num
```

For the given grammar,

(i) Write the pseudocode for building a parse tree.

(ii) Draw the parse tree for the input given below:

input:

```
for ( counter = 1; counter < 5; counter = counter + 1) {
   sum = sum + counter;
}
```

c. Write down the algorithm for removing left recursion. Eliminate the left recursion of the grammar given below showing the steps of the algorithm. **10**

$S \rightarrow A\mathbf{a} \mid B\mathbf{b} \mid \mathbf{b}$
$A \rightarrow A\mathbf{c} \mid S\mathbf{d}$
$B \rightarrow B\mathbf{d} \mid S\mathbf{fg}$

Question – 4

a. Describe the different levels of error handling of a compiler. Explain how the production rules of grammar are represented in YACC / Bison with an example. **04+06 =10**

b. Determine whether the following grammars are LL(1) or not with an explanation. **02*2.5 =05**

(i) $S \rightarrow AS\mathbf{ab} \mid B, A \rightarrow \mathbf{a} \mid \in, B \rightarrow \mathbf{bb}$

(ii) $S \rightarrow A\mathbf{a}A\mathbf{b} \mid B\mathbf{b}B\mathbf{a}, A \rightarrow \in, B \rightarrow \in$

c. Consider the following grammar: **03*05 =15**

$S \rightarrow \mathbf{a}A\mathbf{b}B \mid \mathbf{b}A\mathbf{a}B \mid \in$
$A \rightarrow S \mid \mathbf{e} \mid D\mathbf{a}$
$B \rightarrow S \mid \mathbf{ca}$
$D \rightarrow \mathbf{d} \mid \mathbf{g}$

(i) Identify the **FIRST** and **FOLLOW** set for the grammar given above [Show the FOLLOW table]

(ii) Construct a predictive parsing table using the FIRST and FOLLOW set.

(iii) Show how the string '**agabca**' will be parsed by the non-recursive parser. [Show the stack content, action at each step and other necessary information].

SECTION-B

Question – 5

a. Design the S-attributed SDD which constructs syntax trees for a simple expression grammar involving only the binary operators '+' and '-'. Then find out the parse tree and syntax tree for b–5+p. **05*03 =15**

b. What are the differences between synthesized and inherited attributes for nonterminals? "All S-attributed definitions are L-attributed, but not all L-attributed definitions are S-attributed."- explain and justify. **05+10 =15**

Question – 6

a. Construct the syntax tree and DAG for the following expression: ((x + y) – ((x + y) * (x – y))) + ((x+ y)) * (x – y)) **05*02 =10**

b. Use value- number method to store the DAG in Question 6(a), in an array of records. Then represent the same DAG using three address code. **05*02 =10**

c. What are the disadvantages of triples representation over quadruples representation? Explain how indirect triples overcome the disadvantages of triples. **05+05 =10**

Question – 7

a. Analyze the following C code in Code 1. **08*02 =16**

```
void f ( int n ) {
    int t, s;
    if ( n > 1 ) {
        s = f ( n – 1 );
        t = f ( n – 2 );
    }
}

main ( ) {
    f ( 5 );
}
```

Code 1 : Code for question 7(a)

Now find out the sequence of calls and returns that result from an execution of the above program. Then construct the complete activation tree for it.

b. Define activation record. Why activation record is used? Describe the contents of an activation record with necessary figures and examples. **02+03+09 =14**

Question – 8

a. What are the issues that need to be considered during design of a code generator? Explain the factors of instruction selection with examples. **05+10 =15**

b. Translate the following three address code as shown in Code 2 into an equivalent machine code by using static allocation for procedures a, b and c. Assume that codes for procedures a, b and c start at addresses 3000, 4000 and 5000 respectively. Also assume that activation records for procedures a, b and c start at addresses 7000, 8000 and 9000 respectively. Further assume that each ACTION instruction takes 64 bytes. **15**

```
// code for a
  action1
  action2
```

```
  action3
  call b
  action4
  halt

// code for b
  action5
  action6
  call c
  return

// code for c
  action7
  action8
  return
```

Code 2 : Code for Question 8(b)

BANGLADESH UNIVERSITY OF PROFESSIONALS
Military Institute of Science and Technology
B.Sc. in Computer Science and Engineering
Student Group: 72 < Earned Credit Hour <= 108, Final Examination (Spring): Feb 2021

Subject: CSE-303, Compiler

Total: 2.00 hours **Full Marks: 180**
Section A: 1.00 hour **Section A: 90**

INSTRUCTIONS:

a. Use **SEPARATE** answer scripts for each section.
b. **Question – 1 and Question – 4 (Viva Voce)** in **Section A** are compulsory.
c. Answer any **OTHER ONE** question from this section **(From Q - 2 & Q - 3)**.
d. Figures in the margin indicate full **marks**.
e. Assume reasonable data if necessary.
f. **Symbols** used have their usual meanings.

SECTION-A

Question – 1 (Compulsory)

a. Explain clearly the difference among the following regular expressions in the context of Lexical Analyzer generator: **10**
   i) {cse}{mist}
   ii) [csemist]
   iii) csemist
   iv) (csemist)
   v) $csemist^{+}$

b. Explain the terms token, pattern and lexeme with necessary examples. Which of these is (are) specified in lexical analyzer? How? Explain the role of attribute in relation to token, pattern and lexeme. **10**

c. Left Factor the grammar, **10**

$A \rightarrow abB \mid abcC \mid abcdD$
$B \rightarrow b$
$C \rightarrow c$
$D \rightarrow d$

Question – 2

a. Explain and justify the comment "Java language processors combine compilation and interpretation" **10**

b. Explain how the various stages in a compiler can be grouped into the analysis and synthesis by the symbol table in this scenario? **10**

c. Explain what would have happened if the task of removing white spaces from input programs was moved from lexical analyzer to syntax analyzer? **10**

Question – 3

a. We have the grammar **15**

$$S \rightarrow A * a$$
$$A \rightarrow A + a \mid b$$

Eliminate left recursion from the above grammar. And then find FIRST and FOLLOW for the non-terminals. Use this information to construct the predictive parsing table.

b. Using the predictive parsing table found in the above question (3-(a)), show in a tabular form, the input matched, stack contents and parsing action for an input string, b + a + a * a. From the parsing output, show the derivation of the output string. **15**

Question – 4 (Compulsory) **30**
Marks allotted for Viva 30

BANGLADESH UNIVERSITY OF PROFESSIONALS
Military Institute of Science and Technology
B.Sc. in Computer Science and Engineering
Student Group : 72 < Earned Credit Hour <= 108, Final Examination (Spring) : Feb 2021

Subject: CSE-303, Compiler

Total: 2.00 hours **Full Marks: 180**
Section B : 1.00 hour **Section B : 90**

INSTRUCTIONS:

a. Use **SEPARATE** answer scripts for each section.
b. **Question – 6 and Question – 8 (Viva Voce)** in **Section B** are compulsory.
c. Answer any **OTHER ONE** question from this section **(From Q - 5 & Q - 7)**.
d. Figures in the margin indicate full **marks**.
e. Assume reasonable data if necessary.
f. **Symbols** used have their usual meanings.

SECTION-B

Question – 5

a. Present the SDD which uses inherited attributes to multiply integers. Use this SDD to construct the parse tree for the expression "5 * 6 * 7", annotate the parse tree by calculating the values of the attributes at the various nodes. Then construct the dependency graph for the same expression "5 * 6 * 7". Carry out topological sorting of this graph. **05 x 04 =20**

b. Describe how the memory is subdivided into code and data areas by C++ language compiler for Linux operating system with necessary figures. **10**

Question – 6 (Compulsory)

a. Find out the three-address code and quadruples representation for the following expression: **05 x 02 =10**

n = f(a, g(a, b, c), h(p, q))

b. Compare between name equivalence and structural equivalence with necessary examples. **10**

c. Construct the syntax tree and DAG for the following expression: **05 x 02 =10**

$a * (b + c) + (b + c) * c + a(b + c)$

Question – 7

a. Present the grammar that generates a sequence of declarations with list of names of basic types, array types or record types. Then use this grammar to construct the parse tree for "record {int a, b; float a[3][4];}". **09**

b. Find out the machine instructions for the following three-address statements: **04 x 03 =12**

i. if a<b goto L
ii. b[p] = d
iii. y = * z

c. Explain clearly the difference among, **03 x 03 =09**

i. LD R1, 100 (R2)
ii. LD R1, #100
iii. LD R1, * 100(R2)

Question – 8 (Compulsory) **30**

Marks allotted for Viva 30

BANGLADESH UNIVERSITY OF PROFESSIONALS

Military Institute of Science and Technology

B.Sc. in Computer Science and Engineering, Term Final (Spring) Examination 2021: September 2021

Student Group: 72 < Earned Credit Hours ≤ 108

Subject: CSE-303, Compiler

Total : 3.00 hours (For Physical Examination)

Full Marks : 180
Section A : 90

INSTRUCTIONS - PHYSICAL EXAMINATION

a. **Use SEPARATE answer scripts for each section.**
b. **Question - 1** in **Section A** is compulsory.
c. Answer any **OTHER TWO** questions from **Section-A**.
d. Figures in the margin indicate full **marks.**
e. Assume reasonable data if necessary.
f. **Symbols** and **abbreviations** used have their usual meanings.

SECTION-A

Question – 1 (Compulsory)

a. We have the following grammar for variable declaration in C: **5+10+5 =20**

V → TD;
T → int | float | char
D → D, F | F
F → id

i. Now, eliminate left recursion from the above grammar (if there is any).
ii. Find FIRST and FOLLOW for each non terminal.
iii. Construct the predictive parsing table using the above information.

b. Find out whether the following statements are syntactically correct or not for the above mentioned grammar in Question-1(a) using top down parsing approach. **10**

i. Char id, id;
ii. int id = 4;

If syntactically correct, using the predictive parsing table derived from the above question-1(a), show in a tabular form the input matched, stack content, parsing output sequentially.

Question – 2

a. funDef → typeSpec ID (parms) stmt | ID (parms) stmt **16**

parms ⇒ parmTypeList | ε

parmTypeList →typespec parmIdList

parmIdList → parmIdList, parmTypeList | ID

Stmt → expr; | ;

expr → term = term | term relop term

term → ID | NUMBER

For the above function formulate the regular definition and show the transition diagram for these: identifier, number, semicolon, comma, assignment operator, relational operator and show the transition diagram for each token.

b. "If we do not store the reserved keyword in the symbol table, then we must prioritize the keyword tokens so that the reserved-word tokens are recognized in preference to id, when the lexeme matches both patterns." - support or refuse the statement with justification. **6**

c. Describe where sentinels are used in buffer pairs in lexical analysis. How does the use of sentinels speed up the input process? **8**

Question – 3

a. "Compiler works in two steps whereas interpreter works in one step" – How does this make a difference in working principle of both compiler and interpreter? **5**

b. Explain and justify the comment that Java language processors combine compilation and interpretation. **5**

c. Explain whether the following grammars are LLCD or not, your answer will not be accepted without proper explanation. **20**

i. $S \rightarrow AB \mid Bc \mid d,\ A \rightarrow a \mid \varepsilon,\ B \rightarrow b \mid \varepsilon,\ C \rightarrow c \mid \varepsilon$
ii. $S \rightarrow A \mid a,\ A \rightarrow a$
iii. $S \rightarrow \bar{a}B \mid \bar{\varepsilon},\ B \rightarrow \bar{b}C \mid \bar{\varepsilon},\ C \rightarrow \bar{e}S \mid \bar{\varepsilon}$
iv. $S \rightarrow A,\ A \rightarrow Bb \mid Cd,\ B \rightarrow aB \mid \varepsilon,\ C \rightarrow cC \mid \varepsilon$

Question – 4

a. What should a diagram's code do if the character read at one point is not among the ones for which a following state has been specified? **05**

b. During construction of a parsing table M, for each production $A \rightarrow \alpha$ of the grammar, if ϵ is in FIRST (α), we add $A \rightarrow \alpha$ to M [A, b], for each terminal b in FOLLOW (A). **10**
Explain clearly in light of parsing mechanism.

c. What is intermediate code generation? List two important properties it should have. Give the transition process for statement given below: **05**

$$result = a + b * c + (d/2):$$

d. Preform left factoring in the following grammar: **10**

i. $S \rightarrow a \mid ab \mid abc \mid abcd$
ii. $S \rightarrow bSSāāS \mid bSSāSb \mid bSb \mid ā$

BANGLADESH UNIVERSITY OF PROFESSIONALS

Military Institute of Science and Technology

B.Sc. in Computer Science and Engineering, Term Final (Spring) Examination 2021: September 2021

Student Group: 72 < Earned Credit Hours ≤ 108

Subject: CSE-303, Compiler

Total : 3.00 hours (For Physical Examination)

Full Marks : 180
Section B : 90

INSTRUCTIONS - PHYSICAL EXAMINATION

a. **Use SEPARATE answer scripts for each section.**
b. **Question - 5 in Section B is compulsory.**
c. Answer any **OTHER TWO** questions from **Section-B**.
d. Figures in the margin indicate full **marks.**
e. Assume reasonable data if necessary.
f. **Symbols** and **abbreviations** used have their usual meanings.

SECTION-B

Question – 5 (Compulsory)

a. Draw dependency graph for the syntax directed definition shown in figure-5(a) for the declaration, **10**

float a, b, d;

	PRODUCTION	SEMANTIC RULES
1)	$D \rightarrow T\ L$	$L.inh = T.type$
2)	$T \rightarrow$ **int**	$T.type =$ integer
3)	$T \rightarrow$ **float**	$T.type =$ float
4)	$L \rightarrow L_1$, **id**	$L_1.inh = L.inh$ $addType(\textbf{id}.entry, L.inh)$
5)	$L \rightarrow$ **id**	$addType(\textbf{id}.entry, L.inh)$

Figure-5(a): SDD of Declaration

b. Using the incremental syntax directed translation shown in figure-5(b), draw an annotated parse tree and write the generated three address code for the expression, **10**

m = (n + 10) + (-p + 8)

$S \rightarrow \textbf{id} = E\ ;$ { $gen(top.get(\textbf{id}.lexeme)\ '='\ E.addr);$ }

$E \rightarrow E_1 + E_2$ { $E.addr =$ **new** $Temp();$
$gen(E.addr\ '='\ E_1.addr\ '+'\ E_2.addr);$ }

| $- E_1$ { $E.addr =$ **new** $Temp();$
$gen(E.addr\ '='\ '$**minus**$'\ E_1.addr);$ }

| $(\ E_1\ )$ { $E.addr = E_1.addr;$ }

| **id** { $E.addr = top.get(\textbf{id}.lexeme);$ }

Figure-5(b) Incremental SDT for Code Generation

c. Generate machine code for the following sequence assuming that n, s, i are in a memory location. **10**

```
    s = 0
    i = 0
L1: if i>n goto L2
    s = s + i
    i = i + 1
    goto L1
L2:
```

Question – 6

a. Write three address code and Quadruples for the following code segment using the position number for labels assuming each data takes 4 bytes. **18**

```
a[i] = b * c – b * d
x = f(y + 1) + g(5, f(y + 3))
q = *p + &y
```

b. Write short note on the following concepts: **12**

i. Type Equivalence
ii. Type Expression
iii. Value-number method
iv. DAG

Question – 7

a. Explain the different addressing modes for a simple target machine model. **5**

b. Explain the way of calculating program cost. Calculate program cost for the following target machine code. **10**

```
LD R0, P
LD R1, x
ST O(R0), R1
LD R0, x
LD R1, y
SUB R0, R0, R1
BLTZ *R3, R0
```

c. For the following code, generate machine code using stack allocation for procedures proc1, proc2, and proc3. Assume that code for procedures proc1, proc2 and proc3 start at the address 200, 400 and 600 respectively. Also assume that activation records size for procedures proc1, proc2 and proc3 are 50, 20 and 30 respectively. Further assume that each action instruction takes 32 bytes and stack starts at address 740. **15**

```
//code for proc1
action1
action2
call proc2
action3
action4
halt

//code for proc2
action 5
call proc3
return

//code for proc3
action6
action7
return
```

Question – 8

a. "Stack allocation would not be feasible if procedure calls, or activations of procedures did not nest in time" – Justify the statement. **07**

b. Explain the elements of a general Activation Record. **08**

c. Consider the following pseudo code for generating Fibonacci numbers. Show the downward growing stack for the activation records considering $f(5)$ is called initially and currently $f(1)$ is activated. **15**

```
int f (int n){
      int t, s;
      if (n < 2) return 1;
      s = f(n-1);
      t = f(n-2);
      return s + t;
}
```

BANGLADESH UNIVERSITY OF PROFESSIONALS
Military Institute of Science and Technology
B.Sc. in Computer Scienceand Engineering
Term Final (Spring) Examination 2023: Jul-Sep2023

Student Group: 72<Earned Credit Hours≤108

Subject:CSE 303, Compiler

Time: 3.00 hours **Full Marks: 180**

INSTRUCTIONS:

a. Use **SEPARATE** answer scripts for each section.
b. **Question-1**in **Section-A** and **Question-5** in **Section-B**are compulsory
c. Answer any other**TWO** questions out of **THREE** from each section.
d. Figures in the margin indicate full **marks**.
e. Assume reasonable data if necessary.
f. **Symbols**and **abbreviations** used have their usual meanings.

SECTION-A

Question 1 (Compulsory) **Marks**

a. **10**

L-> D | L D
D -> T I ;
T-> **int** | **float** | **char**
I -> **id** | I **,** **id**

Consider the above grammar for identifier declaration. Based on this grammar, determine whether it is suitable for a top-down or bottom-up parser.
If the grammar is suitable for bottom-up parsing, then apply the shift-reduce process and build bottom-up parser for the following statements**:**

int id1**;** float id2**;**

If not, justify your answer by discussing the characteristics of the grammar.

b. (i) Inspect the grammar of Question-1(a) and determine whether you need to modify the grammar before constructing a predictive top-down parser or not? Justify your response. If so,make the necessary changes and use the modified grammar for the following questions**;** otherwise**,** use the given grammar. **4+8+8**

(ii) Find FIRST and FOLLOW for each non-terminal.

(iii)Construct a predictive parsing table using the FIRST and FOLLOW set.

Question 2

a. Discuss the trade-offs between using a compiler and an interpreter. **10**

b. Compiler is a language processing system which converts a source code to a target machine code. Now, demonstrate the translation done by each phases of the compiler for the following statement considering all the variables are integer numbers. **20**

result = - num1 * 10 + num2 * 40

Question 3

a. Suppose**,** a situation arises where the lexical analyzer is unable to proceed because none of the patterns for tokens matches a prefix of the remaining input. How will the lexical analyzer recover from this phase? **10**

b. Consider the following grammars and patterns for identifying tokens in a programming language **:** **10**

expr -> *term operator term* | **id** *assignment_op term*
term -> **num** | **id**
operator -> **+** | **-** | ***** | **/**
assignment_op -> **=** | **+=** | **-=** | ***=** | **/=**

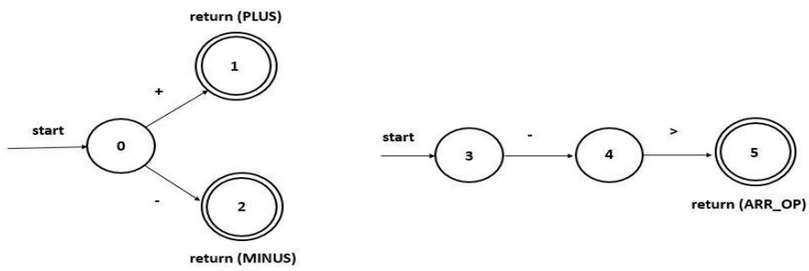
For the given grammar, write down the patterns for identifying the tokens **"**number**",** **"**identifier**",**"operator**"** ,"assignment-op". Also draw the transition diagrams for each of the patterns.

(i) **Number:** Consider of any non-zero number of digits(0-9) but starting digit cannot be zero (0). Decimal points are followed by any non-zero number of digits. This part is optional. There must be at least one digit before the decimal number. Exponential part not considered.

(ii) **Identifier:** Identifier can contain any number of letters or digits or underscore (_).But it cannot start with any digit.

(iii) **Operators** and **assignment operators:** Operators mentioned in the grammar.

c. Consider the following two transition diagrams for arithmetic operator (**'+'** and **'-'** only) and arrow operators (**'->'**) for C language respectively : **10**



If both the transition diagrams are matched in parallel, then how to resolve the case where both the diagrams will reach to the accepting states for **'->'** input?

Question 4

a. Explain whether the following grammars are LL(1) or not; your answer will not be accepted without proper explanation. **10**

i. S-> XX , X->xX|y
ii. S-> ASab|B, A-> a| ∈ , B->bb

b. Explain the back-tracking procedure of recursive-descent top-down parser with proper algorithm and example. **10**

c. Perform left-factoring in the following grammar. **10**

i. F-> n | n [A] | n(A)
ii. S-> bSS' | a ; S' -> SaaS | SaSb |b

SECTION-B

Question 5 **(Compulsory)**

a. Figure – 5(a) represents a Syntax-Directed Definition (SDD) to keep track of block information in the programs. Identify the class of the SDD shown in Figure-5(a) and justify your answer. **07**

PRODUCTION	SEMANTIC RULES
Ss → S ; Ss_1	S.*block* = Ss.*block* Ss_1.*block* = Ss.*block*
Ss → ε	
S → B	B.*block* = S.*block*
B → { Ss }	Ss.*block* = child (B.*block*)
S → **other**	**other**.*block* = S.*block*;

Figure-5(a): Syntax Directed Definition (SDD) for Block information

b. Following grammar generates a binary number with a decimal point. Design a L-attributed SDD to compute the decimal value of a binary string generated by the grammar and using the designed SDD illustrate that, the value of binary string 101.110 is the decimal number 5.75. **15**

S -> L . L | L
L -> L B | B
B -> 0 | 1

c. Appling the Syntax-Directed Definition (SDD) shown in Figure-5(c), build a syntax tree represented as a record for the string **"**b – (a + 3 -5)**"** and list down the steps of tree construction. **08**

PRODUCTION	SEMANTIC RULES
E → E_1 + T	E.*node* = **new** *Node*('+', E_1.*node*, T.*node*)
E → E_1 - T	E.*node* = **new** *Node*('-', E_1.*node*, T.*node*)
E → T	E.*node* = T.*node*
T → (E)	T.*node* = E.*node*
T → **id**	T.*node* = **new** *Leaf* (**id**, **id**.entry)
T → **num**	T.*node* = **new** *Leaf*(**num**, **num**.*val*)

Figure-5(c): Syntax Directed Definition (SDD) for Syntax Tree

Question 6

a. Considering the Syntax-Directed Definition (SDD) in Figure-6(a), build an annotated parse tree and corresponding dependency graph for the declaration, " float arr[4][5][2] ; " . **15**

PRODUCTION	SEMANTIC RULES
T → B **id** C **;**	T.*t* = C.*t* C.*b* = B.*t*
B → **int**	B.t = integer
B → **float**	B.t = float
C → **[num]** C_1	C.t = array (**num**.*val*, C_1.*t*)
C → ε	C_1.*b* = C.*b* C.*t* = C.*b*

Figure-6(a): Syntax Directed Definition (SDD) for type checking

b. Briefly explain the elements of a general activation record for procedure call with appropriate examples. **07**

c. Explain how nonlocal data on the stack can be accessed using the access control link of the activation record with appropriate examples. **08**

Question 7

a. Illustrate three address code for the code segment shown in Code7(a) which turns a 10*10 matrix **A** into an identity matrix. Assume that, the real-valued array elements take 8 bytes each and the matrix A is stored in row-major form. **10**

```
for i from 1 to 10 do
    for j from 1 to 10 do
        A[i,j] = 0.0;
for i from 1 to 10 do
    A[i,i] = 1.0;
```

Code-7(a): Creating Identity Matrix-A

```
// code for procedure proc1
action1
call proc2
action2
halt

// code for procedure proc2
action3
call proc2
return
```

Code-8(a): Pseudo Code

b. Partition the three address instructions that you have generated in question-7(a) into a set of basic blocks and construct flow graph. **10**

c. Compare among the following data structure for representing intermediate representations: **10**

(i) Quadruples
(ii) Triples
(iii) Indirect Triples

Question 8

a. For the three address code shown in Code-8(a), write down generated machine code using static allocations for procedure **proc1** and **proc2**. Use pseudo instruction **ACTION** to represent the sequence of machine instructions to execute the statement **action1**, **action2** and **action3**.Start the code for procedure **proc1** at address **200** and **proc2** at address **350**.Furthermore assume that each ACTION instruction takes **16 bytes** and the activation records for these procedures are statically allocated starting at locations **400** and **494,** respectively. Consider that **action3** has a condition which is responsible for returning back to the caller procedure. You also have to explain the reasons behind each line of the generated machine code. **15**

b. Explain the following optimization mechanism with appropriate examples : **15**

(i) Basic Block Optimization
(ii) Peephole Optimization

BANGLADESH UNIVERSITY OF PROFESSIONALS
Military Institute of Science and Technology
B.Sc. in Computer Science and Engineering, Term Final (Spring) Examination 2024: October - November2024

Student Group: 72<Earned Credit Hours ≤ 108

Subject: CSE 303, Compiler

Time: 3.00 hours **Full Marks: 180**

INSTRUCTIONS:

a. Use **SEPARATE** answer scripts for each section.
b. **Question-1** in **Section-A** and **Question-5** in **Section-B** are compulsory.
c. Answer any other **TWO** out of the remaining **THREE** questions from each section.
d. Figures in the margin indicate full **marks**.
e. Assume reasonable data if necessary.
f. **Symbols** and **abbreviations** used have their usual meanings.

SECTION-A

Question 1 (Compulsory)

a. We have the following grammar for regular expressions over symbols **a** and **b** only: **05+12+07**

rexpr → rexpr + rterm | rterm
rterm → rterm rfactor | rfactor
rfactor → rfactor * | rprimary
rprimary → a | b

(i) Eliminate left recursion from the above grammar (if there is any)

(ii) Now find FIRST and FOLLOW for each non-terminal

(iii) Construct the predictive parsing table using the above information.

b. Using the predictive parsing table derived from Question 1(a), identify whether the following statement is syntactically correct or not. **06**

b * a + ab

Question 2

a. Explain and justify the comment "Java language processors combine compilation and interpretation". **10**

b. Compiler is a language processing system which converts a source code to a target machine code. Now demonstrate the translation done by each phases of the compiler for the following statement considering all the variables are integers. **20**

result = num1 * 10 + num2 * 40

Question 3

a. Consider the following grammar. Write down the patterns for identifying the tokens (mentioned in the question no 3(a)) and draw the transition diagram for each pattern. **15**

stmt → for (expr; expr; expr)
expr → term assignment_operator term
| term increment_operator
| term relop term

term → id | number

Now, write down patterns for number, ID, parenthesis, assignment_operator and increment_operator. Draw transition diagram for the mentioned patterns.

b. Identify the purpose of using two-buffer scheme instead of one-buffer scheme in the process of lexical analysis. **07**

c. Explain clearly the difference among the following regular expressions in the context of Lexical Analyzer Generator **08**

i) [spring 2024]
ii) {spring}{2024}
iii) (spring 2024)
iv) spring 2024

Question 4

a. Explain the back-tracking procedure of recursive-descent top-down parser with proper example. **10**

b. Explain whether the following grammers are LL(1) or not; your answer will not be accepted without proper explanation. **10**

S → XxXy | YyYx
X → xX | ϵ
Y → yY | ϵ

c. Perform left-factoring for the following grammars. **10**

i) S → S+S | SS | (S) | S * | a
ii) S → bSSaaS | bSSasb | bsb | a

SECTION-B

Question 5 **(Compulsory)**

a. The SDD in Fig. 5(a) takes a simple declaration D consisting of a basic type T followed by a list L of identifiers. Semantic rules of the SDD enters the type for each of the identifiers on the list into the symbol-table. **10**
Now, modify the semantic rules in such a way so that it can calculate the width (in bytes) of the identifiers and enter it into the symbol-table. Assume the required attributes to do the task and give proper explanation for the addition of each attribute, rule or side –effect in the semantic rules column.

	PRODUCTION	SEMANTIC RULES
1)	$D \rightarrow T\ L$	$L.inh = T.type$
2)	$T \rightarrow$ **int**	$T.type =$ integer
3)	$T \rightarrow$ **float**	$T.type =$ float
4)	$L \rightarrow L_1$, **id**	$L_1.inh = L.inh$ *addType*(**id**.*entry*. $L.inh$)
5)	$L \rightarrow$ **id**	*addType*(**id**.*entry*. $L.inh$)

Fig. 5(a): Syntax-directed definition for simple type declarations

b. Fig. 5(b) is a syntax-directed definition (SDD) for constructing syntax trees during top-down parsing using the SDD of Fig. 5(b) answer the followings: **20**

i) Identify the class of the SDD with appropriate explanation
ii) Draw parse tree for the expression,
(x+y) – (a-(4+b))
iii) Construct dependency graph for the same expression and
iv) Construct syntax tree using the semantic rules showing the node construction steps of the tree.

	PRODUCTION	SEMANTIC RULES
1)	$E \rightarrow T\ E'$	$E.node = E'.syn$ $E'.inh = T.node$
2)	$E' \rightarrow +\ T\ E'_1$	$E'_1.inh =$ **new** *Node*('+'. $E'.inh, T.node$) $E'.syn = E'_1.syn$
3)	$E' \rightarrow -\ T\ E'_1$	$E'_1.inh =$ **new** *Node*('−', $E'.inh, T.node$) $E'.syn = E'_1.syn$
4)	$E' \rightarrow \epsilon$	$E'.syn = E'.inh$
5)	$T \rightarrow (\ E\ )$	$T.node = E.node$
6)	$T \rightarrow$ **id**	$T.node =$ **new** *Leaf*(**id**, id.*entry*)
7)	$T \rightarrow$ **num**	$T.node =$ **new** *Leaf*(**num**. num.*val*)

Fig. 5(b): Syntax-directed Definition for Constructing syntax trees during top-down parsing

Question 6

a. Explain how indirect triples representation of three address code overcomes the problem of triples representation with appropriate example. **10**

b. The translation scheme in Fig. 6(b) generates three address code for expressions with array references. Considering the SDT given in Fig. 6(b) draw an annotated parse tree for the array referenced expression, **20**

X = arr[i] + st[i][j+1];

and find out the generated three address code using the semantic actions assuming that **arr** is an array of integers and **st** is an array of floating numbers and **X**, **i**, **j** all are integers.

```
S → id = E ;     { gen(top.get(id.lexeme) '=' E.addr); }
  | L = E ;      { gen(L.array.base '[' L.addr ']' '=' E.addr); }
E → E1 + E2      { E.addr = new Temp ();
                   gen(E.addr '=' E1.addr '+' E2.addr); }
  | id           { E.addr = top.get(id.lexeme); }
  | L            { E.addr = new Temp ();
                   gen(E.addr '=' L.array.base '[' L.addr ']'); }
L → id [ E ]     { L.array = top.get(id.lexeme);
                   L.type = L.array.type.elem;
                   L.addr = new Temp ();
                   gen(L.addr '=' E.addr '*' L.type.width); }
  | L1 [ E ]     { L.array = L1.array;
                   L.type = L1.type.elem;
                   t = new Temp ();
                   L.addr = new Temp ();
                   gen(t '=' E.addr '*' L.type.width);
                   gen(L.addr '=' L1.addr '+' t); }
```

Fig. 6(b): Semantic actions for array references

Question 7

a. Briefly explain the elements of a general activation record for the procedures with appropriate examples. **07**

b. Fig. 7(b) is a C code to compute n^{th} Fibonacci numbers recursively. Suppose that the activation record for the function **fibo** includes the following elements in order: (return value, argument n, local s, local t); there will normally be other elements in the activation record as well. To answer the questions below assume that the initial call is **fibo(5)**. **15**

i) Draw the complete activation tree for the call fibo(5) by simulating the program given in Fig. 7(b).

ii) What does the stack and its activation records look like the first time fibo(1) is about to return?

iii) What does the stack and its activation records look like the fifth time fibo(1) is about to return?

```
int fibo (int n){
   int t, s;
   if (n<2) return 1;
   s = fibo(n-1);
   t = fibo(n-2);
   return s+t;
}
```

Fig. 7(b): Program to compute n^{th} Fibonacci Number

c. Explain how non-local data on the stack can be accessed using the access control link of the activation record for a language which supports nested procedure declaration and procedure can have function-valued parameter with appropriate examples. **08**

Question 8

a. For a simple target machine explain how program cost can be estimated for different types of instruction. **10**
Now determine the total program cost of the following instruction sequences:

```
LD R0, y
LD R1, #10
ADD R0, R0, R1
INC R0, R0
LD R1, 200(R0)
ST x(R0), R1
BLTZ *R0, R1
```

b. For the pseudo-code given in Fig. 8(b), considering the instructions of a simple target machine, generate machine code using stack allocation for procedures main, proc and add. Assume that code area for procedures main, proc and add start at the addresses 100, 200 and 500 respectively. Also assume that activation records size for the procedures main, proc and add are 20, 40 and 10 respectively. Further assume that, each action1, action2, action5 takes 16 bytes and each action3, action4, action6 and action7 instruction takes 10 bytes and the stack starts at address 740. Consider that action7 has a condition which is responsible for returning back to the caller procedure. Assume the registers and instructions as required for the target machine. **20**

```
// code for main procedure

action1
action2
call proc
action3
halt

// code for proc procedure

action4
call add
action5
call proc
action6
call proc
return

// code for add procedure

action7
action4
return
```

Fig. 8(b): Psuedo-code for a simple target machine.

BANGLADESH UNIVERSITY OF PROFESSIONALS
Military Institute of Science and Technology
B.Sc. in Computer Science and Engineering, Term Final (Spring) Examination 2025: Oct - Nov 2025

Student Group: 72<Earned Credit Hours ≤ 108

Subject: CSE-303, Compiler

Time: 3.00 hours **Full Marks: 180**

INSTRUCTIONS:

a. Use **SEPARATE** answer scripts for each section.
b. **Question-1**in **Section-A** and **Question-5** in **Section-B** are compulsory.
c. Answer any other **TWO** out of the remaining **THREE** questions from each section.
d. Figures in the margin indicate full **marks**.
e. Assume reasonable data if necessary.
f. **Symbols** and **abbreviations** used have their usual meanings.

SECTION-A

Question 1 **(Compulsory)**

Consider the following grammers and answer the Questions.

Grammer-01 (G-01):
$E \rightarrow E + E \mid E * E \mid (E) \mid id \mid num$

Grammer-02 (G-02):
$E \rightarrow E + T \mid T$
$T \rightarrow T * F \mid F$
$F \rightarrow (E) \mid id \mid num$

a. Identify the set of lexemes that these above grammers can generate. Define suitable regular expressions (REs) for each token. **8**

b. Describe the problems with grammer G-01 and how does G-02 resolve these problems with appropriate example and parse tree. **10**

c. Can grammer G-02 be used for a top-down parser (recursive descent)? Justify your answer. If no, transform G-02 into an equivalent grammer that is suitable for top-down parsing. **12**

Question 2

Consider the following grammer:
$S \rightarrow ACB \mid Cbb \mid Ba$
$A \rightarrow da \mid BC$
$B \rightarrow g \mid \epsilon$
$C \rightarrow h \mid \epsilon$
where S, A, B and C are non-terminals and a, b, d, g, h and ϵ are terminals.

a. Find the FIRST and FOLLOW for each non-terminals of the above grammer. **8**

b. Build a predictive parsing table from FIRST and FOLLOW set. **14**

c. Explain the concept of Non-Recursive Predictive Parsing. Describe the algorithm used by such a parser to process an input string. **8**

Question 3

a. Explain the role of “Buffer Pair” in the design of a lexical analyzer and how this scheme works. **10**

b. Construct the Regular Expressions (REs) and show the transition diagram that will match Date format. Day, Month, and year – all three parts must be present in the date format. The parts can be separated by either dot (.) or hyphen (-). Year must be written in 4 digit format. Needless to say that the numbers in the date should represent valid numbers. **10**

Example of some correct and incorrect date formats are given below:

Correct:

12-03-2022, 1-2-2000, 1.11.2050

01-10-2019, 01.05.1950, 1-02-2024

Incorrect:

36.9.2022, 3-14-2023, 12.12.22

c. "If we do not store the reserved keywords in the symbol table, then we must prioritize the keyword tokens so that the reserved word tokens are recognized preference to id, when the lexeme matches both pattern". Justify the statement. **10**

Question 4

a. Compiler is a language processing system which converts a source code to a target machine code. Now, demonstrate the translation done by each phases of the compiler for the following statement considering all the variables are integer numbers. **20**

$$result = -num1 * 10 + num2 * 40$$

b. Perform left-factoring in the following grammer. **10**

(i) $F \rightarrow n \mid n[A] \mid n(A)$

(ii) $S \rightarrow bSS' \mid a$

(iii) $S' \rightarrow SaaS \mid SaSb \mid b$

SECTION-B

Question 5 **(Compulsory)**

Answer all the questions for following CFG rules: **5+8+10+7=30**

$L \rightarrow En$

$E \rightarrow E_1+T$

$E \rightarrow T$

$T \rightarrow T_1* F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow digit$

Questions:

a. Define Syntax Directed Translation with importance.

b. Differentiate between Syntax, Directed Definitions and Syntax Directed translation schemes in terms of the above CFG or production rules.

c. Produce the Semantic Rules followed by an annotated parse tree for the expression Input:$5 + 3 * 4$
along with a dependency graph for both synthesized and inherited attributes.

d. For the following expressions and binary operators, draw the respective syntax tree
- mknode (op, left, right)
- mkleaf (id, entry)
- mkleaf (num, val)

Question 6

a. Analyze the Bottom-up evaluation procedure for the expression- $5 + 3 * 4n$ with step by step derivation. **12+6+6+6=30**

b. State the translation schemes for S-attributed definitions with an example.

c. Analyze on requirements of inherited and synthesized attributes in Syntax Directed Definitions.

d. Explain the role of Intermediate Code generations in Compiler Design.

Question 7

a. For the following arithmatic expression, what will be the representation in 3-Access Codes and postfix Notations. **6+6+8+10=30**
$A = B + C * D$

b. Explain how intermediate codes ensures portability and optimization opportunities for hardware architectures.

c. Discuss on the control flows of if-else and while-loop statements with examples drawing Graphs.

d. Illustrate the possible issues of flow controls among different basic blocks of codes given below with respective Directed Acyclic Graphs.
$a = b + c$
$b = a - d$
$c = b + c$
$d = a - d$

Question 8

a. Discuss on the basic requirements of Code-Optimization along with their opportunities. **5+6+5x3+4=30**

b. Define Code-Optimization in the context of necessity on trade-offs between compilation used and execution efficiency for machine dependent and independent optimization.

c. For the following intermediate code sequences
(i) Identify redundant and unnecessary computation.
(ii) Rewrite optimized code using common sub expression elimination and dead code elimination.
$t_1 = a * 2$
$t_2 = a * 2$
$t_3 = t_1 + t_2$
$t_4 = 0 * b$
$t_5 = t_3 + t_4$
(iii) For the following loop, produce the optimized code applying code motion and strength reduction

```
For(i=0; i<10; i++){
   x = 4*i;
   y = 20;
   z = x+y;
 }
```

d. Discuss on the applicability on Peephole Optimization with examples.